UPL — Universal Prompt Language specification

- **Version:** 1.0
- **Status:** Production release
- **File Extension:** `.txt` or `.upl`
- **Changelog:**

```
- 08-09-2026 - v1.0 published - David Valin <hola@davidvalin.com>
- 10-08-2026 - v1.0 draft published - David Valin <hola@davidvalin.com>
```

---

1. Overview

The **Universal Prompt Language (UPL)** is a human-readable language for authoring parameterized self-contained prompt templates. A UPL file declares a set of typed input variables, their defaults and options, plus a prompt body containing placeholders, conditional expressions, and `for` loops. At render time, the engine substitutes variable values, evaluates conditionals with runtime type checking, and expands loops to produce the final prompt string.

UPL is designed to be:

- **Human-readable** — plain text, easy to author and review.
- **Self-contained** — the UPL prompt defines the complete list of parameters, the prompt body, and its rendering logic.
- **Strictly typed** — every variable has an explicit type, validated at parse and render time.
- **Safe** — only the `[[[` and `{{{` delimiters trigger expansion; unmatched blocks are ignored, so code snippets containing similar-looking sequences are not misinterpreted.
- **Composable** — supports nested objects, lists of objects, reusable object shapes (`object_shape`), shape reuse via `type: <object_shape_name>` , and recursive structures.

---

2. File Format

A UPL file is a plain text file with the `.txt` or `.upl` extension. No other extension is permitted. It has two sections — metadata and body — separated by a line containing exactly `--`; the body may optionally end with a line of its own `--` (a delimiter, not a third section):

```
[--; optional]
<metadata section>
--
<prompt body section>
[--; optional, marks end of body]
```

- The **metadata section** is a YAML-like block declaring the prompt's identity and its input variables.
- The **prompt body** is free text that may contain placeholders, conditionals, and loops.
- A leading `--` line is **optional**: a file may begin directly with its first metadata key. A trailing `--` is also optional and conventionally used to mark the end of the body. Sections are separated by a line containing exactly `--`.
- The body ends at the **first** line whose trimmed content is exactly `--` (if any); everything from there to the end of the file is the terminator, not body content. Consequently a bare `--` line on its own — e.g. a divider, a CLI `cmd -- args` example written alone, or a YAML-style separator — can never appear as literal body text; there is no escape for it, so such a line must be avoided (e.g. by adding trailing context to the same line, such as `cmd -- args` with the command on it, rather

than a lone `--`). Any further **non-blank** line found after the terminator is a parse error (trailing blank lines are harmless and ignored) — earlier revisions of this format silently discarded such trailing content instead of rejecting it.

The `name` metadata field (§2.1) MUST be equal to the file's base name (the file name with its `.txt` or `.upl` extension stripped). A single trailing `.prompt` segment — as produced by legacy tooling — is also stripped before the comparison, so both `my_prompt.txt` and `my_prompt.prompt.txt` resolve to the base name `my_prompt`. Any other suffix is not stripped. A mismatch between the `name` field and the file name is a parse/load error.

All values are **case-sensitive**. Variable and field *declarations* in `params` are **lowercase** identifiers, while every *reference* to a variable in the prompt body (placeholders, loop bindings, ternary branches, condition variables) MUST be **uppercase** and resolves case-insensitively against the declared name (see §4.1). Indentation uses **spaces only** (tabs are not permitted).

2.1 Metadata Fields

Field	Required	Description
`name`	Yes	Unique identifier of the prompt. MUST match the file's base name (see §2). Only **lowercase** alphanumeric (UTF-8) characters and underscores (`_`) are allowed; uppercase letters, hyphens, dots, and other punctuation are not permitted. An empty value is invalid.
`title`	No	Human-readable title.
`desc`	No	Optional description of the prompt's purpose.
`source`	No	Provenance field, `<host>/<username>/<prompt_name>`, injected automatically when a prompt is pulled from a UPL repository. Authoring tools SHOULD NOT set it by hand; it is informational and does not affect parsing or rendering.
`params`	Yes	Map of variable declarations (see §3).

`title`, `desc`, and `source` are taken **verbatim**: everything after the `key:` prefix, trimmed of surrounding whitespace only, becomes the value with no quote-stripping or escape processing. Consequently these fields are written **bare** (unquoted) in examples throughout this document — `desc: An example prompt`, not `desc: "An example prompt"`. Wrapping a value in quotes is not an error, but the quote characters become part of the stored value, since they are not stripped. This differs from `def:` literals (§3.3.1), which do use quotes to delimit a string and strip them.

`params` MUST be the **last** metadata field. A parser locates the end of the `params` block purely by indentation (there is no other terminator), so any metadata key written after it is not recognized as metadata — it is read as part of the prompt body instead, along with everything up to the next `--`. Declare `title` / `desc` / `source` (if present) before `params`.

2.2 Example Skeleton

```
--
name: my_prompt
title: My Prompt
desc: An example prompt
params:
  username:
    type: string
    desc: The user's name
    def: "guest"
--
Hello, [[[USERNAME]]]!
--
```

---

3. Variable Definitions

Variables are declared under the `params` block. Each variable has the following fields:

Field	Required	Applies to	Description
`type`	Yes	All	One of the types listed in §3.1.
`desc`	No	All	Optional human-readable description. Taken verbatim, like the metadata fields (§2.1) — written bare/unquoted.
`def`	No	All	Default value used when no value is supplied. For `object_shape`, the declared `def`/field defaults are applied at every site that references the object_shape.
`opts`	No	`option_single`, `option_multi`	List of allowed options. Each entry must match the variable's `etype` (§3.3).
`etype`	Conditional	`list`, `option_single`, `option_multi`	Element type: a built-in type name (§3.1) — including the inline `object` (the variable then declares its element shape via its own `ofields`) — or the name of a declared `object_shape` variable (§3.4). Not allowed on `object` / `object_shape` (an object's shape is described by `ofields`). For `option_single` it is **optional** and defaults to `string`; for `option_multi` it is **required**. The allowed etypes for `option_single` / `option_multi` are `string`, `long_string`, `number`, the inline `object`, and a referenced `object_shape` (§3.4). `boolean`, `list`, `option_single`, `option_multi`, and `object_shape` (the literal type name) are not valid option etypes. The allowed etypes for `list` are `string`, `long_string`, `number`, `boolean`, the inline `object`, and a referenced `object_shape` (§3.4); `list`, `option_single`, `option_multi`, and `object_shape` (the literal type name) are not valid list etypes.
`ofields`	Conditional	`object`, `object_shape`, and `list` / `option_single` / `option_multi` whose `etype` is the inline `object`	Map of object field definitions (recursive). Required on `object_shape`; on `object` either `ofields` (inline) or `type: <object_shape_name>` (§3.4.2) must

Field	Required	Applies to	Description
			be present, but not both. On a `list` / `option_*` it declares the element shape of an inline `etype: object` (§3.3, §4.1.1) — a by-name `etype: <object_shape>` reference takes its shape from the referenced object_shape instead, and must not also declare `ofields`.
`label`	No	`option_single`, `option_multi`	Required whenever `etype` is object-shaped — the inline `object` etype or a referenced `object_shape` alike: the field name (declared on that shape) whose value is shown as the menu label for each option. Not allowed for scalar etypes.
`exclude_condition`	No	All top-level except `object_shape`	A build-time condition expression (§5 syntax) that controls whether the parameter is **shown** or **hidden** during the build. When the condition evaluates to a **truthy** value, the parameter is **hidden** (excluded from the build — not collected, and no override for it accepted, by any value-supply mechanism). When the condition is **falsy** (or absent), the parameter is **shown** (collected normally). A condition may only reference parameters declared *before* the one carrying it (see §3.7).

`def` is **optional** for every type. When `def` is omitted (and no value is supplied interactively or programmatically), the variable falls back to a type-appropriate default:

Type	Default when `def` is absent
`string`	`""`
`long_string`	`""`
`number`	`0`
`boolean`	`false`
`list`	`[]`
`option_single`	the first entry in `opts`
`option_multi`	`[]`
`object`	an object whose each field is set to its own default per this table (recursively)
`object_shape`	not asked at its definition site; its `def`/field defaults are applied where it is referenced (see §3.4)

These fallbacks are also used to synthesize missing nested fields when rendering with defaults.

An `object` / `object_shape`-typed variable may declare **both** an object-level `def` literal and, on its own `ofields` (or the `ofields` of the `object_shape` it reuses via `type: <name>`, §3.4.2), field-level `def`s for individual fields. When both are present, the object-level literal wins **per key**: a field the literal declares takes its value from the literal; a field the literal doesn't mention falls back to that field's own `def` (or its type-appropriate zero value if it has none). This merge is recursive — a nested object field within the literal is itself merged the same way against that nested field's own defaults, rather than replacing the whole nested object wholesale. For example, given a `philosopher` shape whose fields default to `name: "Socrates"` and `era: -470`, declaring `focal: type: philosopher, def: { name: "Plato" }` yields `{ name: "Plato", era: -470 }` — `name` from the literal, `era` from the shape.

3.1 Supported Variable Types

All type names are **lowercase**.

Type	Description	Requires `etype`	Requires `ofields`	Requires `opts`
`string`	Plain string	No	No	No
`long_string`	Multi-line or long string (e.g. code)	No	No	No
`number`	Floating-point number	No	No	No
`boolean`	`true` / `false`	No	No	No
`list`	List of free entered values	**Yes**	Only with an inline `etype: object`	No
`object`	Struct-like object with named fields. Asked to the user as a parameter in declaration order.	No	**Yes** (or `type: <object_shape>`)	No
`object_shape`	Reusable object shape (same fields as `object`). **Not** asked to the user at its definition site; only asked where it is referenced.	No	**Yes**	No
`option_single`	Single choice from a list of options	No — optional, defaults to `string`	Only with an inline `etype: object`	**Yes** (≥ 2)
`option_multi`	Multiple choices from a list of options	**Yes**	Only with an inline `etype: object`	**Yes** (≥ 2)

> A `long_string` variable also accepts a heredoc form for `def` (see §3.5).

> The `etype` of an `option_single` / `option_multi` may be `string`, `long_string`, `number`, the inline `object` (with its own `ofields`), or the name of a declared `object_shape` variable (§3.4). `boolean` is not a valid option etype. Whenever `etype` is object-shaped — inline or a referenced object_shape alike — the `label` field is **required** (§3.6).

> `object_shape` and `object` both describe an object shape via `ofields`, but differ in how they are presented to the user at build time: an `object` is a **collectible parameter** (the builder prompts for its fields in declaration order), while an `object_shape` is a **pure type definition** and is never prompted for on its own — it is only collected at the site that references it (a `list` / `option_*` element, or an `object` inheriting its fields via `type: <name>`). Both `object` and `object_shape` may appear as nested field types; when nested, both are collected inline during their parent's collection (the distinction only matters at root level).

3.2 Nested Objects

An `object` variable is a **collectible parameter**: at build time the builder prompts the user for each of its fields, in declaration order (interleaved with the other top-level params in the order they were declared). Its fields are declared under an `ofields` block. Field definitions may themselves be `object` variables, allowing arbitrary recursion depth.

```
params:
  server:
    type: object
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
      ssl:
        type: object
        ofields:
          enabled:
            type: boolean
            def: true
          cert:
            type: string
            def: ""
```

An `ofields` entry of an `object` (or `object_shape`) may **not** reference a top-level `object` param: there is no `etype: <object_name>` form and no `type: <object_name>` form. To reuse a shape inside an `object` (or `object_shape`) field, declare the shared shape as an `object_shape` and use it via `type: <name>` (§3.4). Nesting of `object` / `object_shape` fields inside `object` / `object_shape` ofields remains fully supported (as in the `ssl` field above).

3.3 Validation Rules

- `type` must be one of the values listed in §3.1.
- `etype` is only allowed when `type` is `list`, `option_single`, or `option_multi`. It is not allowed on `object` or `object_shape` (an object's shape is described by `ofields`).
- `etype` may be either a built-in type name (§3.1) — including the literal `object`, in which case the variable declares its element shape inline via its own `ofields` — or the name of a declared `object_shape` variable (see §3.4). Naming a declared `object` variable (instead of an `object_shape`) via `etype` is an error — only `object_shape` is referenceable by name.
- For `list`, `etype` may only be `string`, `long_string`, `number`, `boolean`, the inline `object` (with its own `ofields`), or a referenced `object_shape`. `list`, `option_single`, `option_multi`, and `object_shape` (the literal type name) are not valid list etypes.
- For `option_single` and `option_multi`, `etype` may only be `string`, `long_string`, `number`, the inline `object` (with its own `ofields`), or a referenced `object_shape`. `boolean`, `list`, `option_single`, `option_multi`, and `object_shape` (the literal type name) are not valid option etypes.
- For `option_single`, `etype` is optional and defaults to `string` when omitted. For `option_multi`, `etype` is required.
- A `list` / `option_single` / `option_multi` whose `etype` is the inline `object` MUST declare its element fields via its own `ofields` block; there is no shape to splice in from elsewhere, so an inline `object` etype with no `ofields` is a parse error.
- `label` is only allowed when `type` is `option_single` or `option_multi`, and is **required** whenever their `etype` is object-shaped — either the inline `object` etype (with its own `ofields`) or a referenced `object_shape` (the by-name form); the two forms follow the same rule. `label` MUST name a field declared on that object shape, and that field's type MUST be `string` or `long_string`. `label` is not allowed (and has no effect) for scalar etypes (`string`, `long_string`, `number`).
- An element reference (the by-name form) MUST resolve to an `object_shape` variable that declares an `ofields` block. Element reference cycles are not allowed and MUST be reported as an error.

- `ofields` is allowed on `object` and `object_shape`, and on a `list` / `option_single` / `option_multi` whose `etype` is the inline `object`. Any other type declaring `ofields` is a parse error. On an `object`, `ofields` is the inline field map; on an `object_shape`, `ofields` is always a map of field definitions (a shape declaration); on a `list` / `option_*` it is the element shape of the inline `etype: object` (and is required in that case — see the next bullet). An `object` must declare either `ofields` (inline) or `type: <object_shape_name>` (reuse a shape), but not both — `type: <name>` and `ofields` are mutually exclusive. Likewise a `list` / `option_*` declares either an inline `etype: object` + `ofields` or a by-name `etype: <object_shape>`, never both.
- `type` accepts either a built-in type name (§3.1) or the name of a declared `object_shape` variable (§3.4.2). A non-builtin `type` value names a declared `object_shape` whose `ofields` are spliced in as this `object`'s own fields; the variable is then an `object` value of that shape (asked at root, collected inline when nested). Naming a declared `object` (instead of an `object_shape`) via `type` is a parse error — only `object_shape` is referenceable by name. Built-in type names are reserved and cannot be used as `object_shape` names.
- An `ofields` entry of an `object` (or `object_shape`) may not reference a top-level `object` param: the only reusable-shape reference forms are `type: <object_shape_name>` (on `object` / `object_shape` fields) and `etype: <object_shape_name>` (on `list` / `option_*`). Referencing an `object` via `type` or `etype` is a parse error.
- `opts` is only allowed for `option_single` and `option_multi`. Any other type declaring `opts` is a parse error. `opts` MUST contain **at least two entries** — a single-option menu is not meaningful and is a parse error.
- All values supplied via `def`, `opts`, etc. must match the declared `type` and `etype`. Specifically, each entry in `opts` MUST be coercible to the option's `etype`: a `string` / `long_string` entry for a `string` / `long_string` etype, a `number` entry for a `number` etype, and an object literal matching the referenced object_shape's `ofields` shape (or the inline `ofields` shape) for an `object` / `object_shape` etype. A mismatch is a parse error.
- The `def` value MUST match the declared `type` (and, for `list` / `option_multi`, its `etype`): a `def` for a `number` variable must be a number literal; a `def` for a `boolean` must be `true` / `false`; a `def` for a `string` / `long_string` must be a string; a `def` for an `object` or `object_shape` must be an object literal; a `def` for a `list` must be a list whose every element matches `etype`; a `def` for an `option_single` must be a single value matching `etype`; a `def` for an `option_multi` must be a list whose every element matches `etype`. A `def` value of the wrong kind is a parse error. (Object `def`s are checked for kind only — `object` — not for full field-shape conformance; extra or missing nested keys are tolerated and filled from defaults at render time.)
- For `option_single` and `option_multi`, the `def` value MUST additionally be one of the declared `opts`: for `option_single` the value itself, and for `option_multi` **every** element of the list. A `def` that is not an offered option is a parse error — the same rule an implementation applies to a value supplied at build time. Object-shaped entries are compared after being completed from the element shape's field defaults (§3.4), so a partially written `def` matches the option it names in full.
- `exclude_condition` (§3.7) is only allowed on top-level (root) parameters — declaring it on a nested `ofields` entry is a parse error. It is not allowed on `object_shape` variables (they are never asked at build time). The condition expression uses the same syntax as body conditions (§5) and is parsed and validated at parse time (§9.5). Variable references inside a condition MUST be uppercase (§4.1) and MUST name a previously-declared top-level parameter; a forward or self-reference is a parse error.

3.3.1 Literal Value Syntax

Values supplied for `def` and `opts` may be written in any of the following forms:

- **Quoted string** — `"..."` or `'...'`. Both styles yield the same value; a quote character that does not start a string literal must be enclosed in the other quote style. Used for `string`, `long_string`, and string-valued `option_*` entries.
- **Bare number** — a floating-point literal such as `80`, `3.14`, or `-1`. Used for `number` and number-valued entries.
- **Bare boolean** — `true` or `false`.
- **Inline list** — `[v1, v2, ...]`, items separated by commas. Items may be any of the forms in this section, including nested lists/objects. Whitespace around items is ignored.
- **Inline object** — `{ key: value, key2: value2, ... }`. Keys are bare identifiers; values are any of the forms in this section. Commas separate entries; nested `{}` / `[]` and string literals are respected, so values may themselves contain commas or colons.
- **Block list** — instead of an inline `[...]`, a `def:` (or `opts:`) line with an empty value may be followed by indented `- <value>` lines (at indent + 4 spaces). Each item is parsed with the rules above.
- **Heredoc** — for `long_string` `def` only, see §3.5.

Bare (unquoted) tokens that are not `true` / `false` /numbers/inline collections are treated as strings.

3.4 Object Type Reuse (Element References)

The `etype` of a `list`, `option_single`, or `option_multi` may name a **previously- or forward-declared `object_shape` variable** instead of a built-in type. The referenced object_shape's `ofields` shape is then reused as the element structure of the list/options, so the field definitions need not be repeated inline. Likewise, an `object` (or a nested object field) may reuse a declared `object_shape`'s `ofields` by writing `type: <object_shape_name>` instead of `type: object` + an inline `ofields` map; the referenced object_shape's fields are spliced in as the object's own fields.

The referenced variable MUST be declared in the same `params` block with `type: object_shape` and an `ofields` block. References are resolved at parse time; after resolution the referencing variable behaves exactly as if the referenced object_shape's `ofields` had been written inline. Forward references (declaring the referencing variable before the object_shape it names) are permitted. Circular references are not allowed and MUST be reported as errors. For `option_single` / `option_multi` with an `object_shape` etype, the `label` field (§3.6) is required.

> An `object_shape` is **not** asked to the user at its definition site — it is a pure type definition. It is only collected at the site that references it: a `list` / `option_*` element prompt, or an `object` /field that uses it via `type: <name>` (in which case that `object` /field is the one prompted, with the shape's fields). A top-level `object` (not `object_shape`) cannot be referenced via `etype` / `type`; only `object_shape` is referenceable. Both `object` and `object_shape` may appear as nested field types; when nested they are collected inline during parent collection (identically), so the `object`-vs-`object_shape` distinction only matters at root level.

3.4.1 `etype` reference (list / option_single / option_multi)

```
params:
  host:
    type: object_shape
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
  servers:
    type: list
    etype: host
    def:
      - { host: "localhost", port: 8080 }
      - { host: "db.local", port: 5432 }
--
Hosts:
{{{for S in SERVERS}}}
- [[[S.HOST]]]:[[[S.PORT]]]
{{{end for}}}
```

renders to:

```
Hosts:
- localhost:8080
- db.local:5432
```

The same mechanism works for `option_multi`, where each chosen option is an object shaped like the referenced `object_shape`:

```
params:
  feature:
    type: object_shape
    ofields:
      name:
        type: string
      enabled:
        type: boolean
        def: false
  selected:
    type: option_multi
    etype: feature
    label: name
    opts:
      - { name: "auth", enabled: true }
      - { name: "logs", enabled: false }
--
Enabled:
{{{for F in SELECTED}}}
- [[[F.NAME]]]
{{{end for}}}
```

The object_shape's field-level `def`s reach every element this way, not just the list/option's own `def:` literal: if an individual element is supplied only partially — e.g. via JSON, `{"servers": [{"host": "only"}]}` against the `servers` declaration above — any field the element's value doesn't mention (here, `port`) falls back to the shape's own field default (`8080`) for that field, the same as it would for a top-level `object`. There is no way to declare a *per-element* object-level default override the way `type: <name>` reuse can (§3.4.2, §3 `def` row) — a list/option has one shared element shape, not a fixed set of individually-declared instances, so only the shape's own field defaults apply uniformly to every element; supplying a different value per element is only possible via `def:`/JSON/interactive input, not via a declared default.

3.4.2 Shape reuse (`type: <object_shape_name>`)

An `object` (or a nested object field) may reuse the fields of a declared `object_shape` by writing `type: <object_shape_name>` instead of `type: object` with an inline `ofields` map. The resulting `object` is still a **collectible parameter** (it is prompted at build time), but its field shape is the referenced object_shape's `ofields`. This is the form to use when an object param (or field) should reuse a shared shape defined once. `type: <name>` is mutually exclusive with `ofields` (the referenced object_shape provides the fields). Built-in type names are reserved and cannot shadow or be shadowed by an `object_shape` name.

```
params:
  host:
    type: object_shape
    ofields:
      host:
        type: string
        def: "localhost"
      port:
        type: number
        def: 8080
  cfg:
    type: host
--
Host: [[[CFG.HOST]]] Port: [[[CFG.PORT]]]
```

Renders (with defaults) to:

```
Host: localhost Port: 8080
```

At build time the builder prompts for `cfg` (not `host`), collecting `host` and `port` as its fields. The `host` object_shape itself is never prompted for on its own.

3.5 Long String Defaults (Heredoc Form)

For `long_string` variables, the `def` value may alternatively be supplied as a **heredoc block**. This is convenient when the default is a long, multi-line text such as a code snippet or a paragraph that is unpleasant to write as a single quoted line with embedded escapes.

The syntax is:

```
def: >>>
raw content lines,
at any indentation
<<<
```

- The value of the `def:` key is the literal token `>>>` (with nothing else on the line).
- Every subsequent line — regardless of indentation — is part of the default value, taken **verbatim** (no escape processing, no quote stripping).
- The block ends at the first line whose trimmed content is exactly `<<<`. That terminator line is consumed and is not part of the value.
- If the end of file is reached before a `<<<` terminator, parsing fails with an error.
- The heredoc form is **only** valid for `long_string` variables. Using it on any other type is a parse error.
- `type: long_string` MUST be declared **before** `def: >>>` within the variable's block. The heredoc check runs against whatever type has been parsed so far, so `def: >>>` appearing first (before `type:` has been seen) fails with the same error as using it on a non-`long_string` type, even though the variable is a `long_string` overall.

The collected lines are joined with `\n` to form the default value; the newline that precedes the `<<<` terminator is not included, so the value is exactly the text between `>>>` and `<<<`.

```
params:
  body:
    type: long_string
    desc: Default JSON request body
    def: >>>
{
  "name": "example",
  "active": true
}
<<<
--
POST [[[URL]]]
Content-Type: application/json

[[[BODY]]]
```

renders — with `BODY` defaulted and `URL` supplied at render time as `https://api.example.com` — to:

```
POST https://api.example.com
Content-Type: application/json

{
  "name": "example",
  "active": true
}
```

3.6 Option Labels (`label`)

When an `option_single` or `option_multi` variable has an object-shaped etype, each entry in `opts` is an object literal. The interactive menu needs a single, human-readable string to display for each option, so a `label` field is **required** in that case — whether the etype is the inline `object` (with its own `ofields`) or a referenced `object_shape` (the by-name form); the requirement is identical either way, since both produce an object-valued option with no built-in string representation. `label` names a field declared on that object shape; the value of that field on each option object becomes its menu label.

The named field MUST be declared on the object shape with type `string` or `long_string`. `label` is only meaningful for `option_single` / `option_multi` with an object-shaped etype (inline or referenced) and is not allowed (and should be omitted) for scalar etypes.

```
params:
  feature:
    type: object_shape
    ofields:
      name:
        type: string
      enabled:
        type: boolean
        def: false
  selected:
    type: option_multi
    etype: feature
    label: name
    opts:
      - { name: "auth", enabled: true }
      - { name: "logs", enabled: false }
--
Enabled:
{{{for F in SELECTED}}}
- [[[F.NAME]]]
{{{end for}}}
```

In the menu, the two options are shown as `auth` and `logs` (the values of the `name` field). The chosen objects are stored whole, so `[[[F.NAME]]]` and `[[[F.ENABLED]]]` resolve normally inside a loop.

3.7 Build-Time Conditions (`exclude_condition`)

A top-level parameter may declare an `exclude_condition` — a condition expression (evaluated with the operators in §5) that controls whether the parameter is **shown** or **hidden** during the build. "Shown"/"hidden" describe the parameter's *value-collection status*, independent of however a given implementation supplies values to a build (interactively, programmatically, or by any other host-defined mechanism — that mechanism's own contract is outside this specification's scope; only its interaction with `exclude_condition` is normative here):

- **Condition is truthy → parameter is hidden.** The parameter is excluded from the build: its declared `def` default is used, and no override for it is collected or accepted by any means while it remains hidden.
- **Condition is falsy (or no `exclude_condition` is declared) → parameter is shown.** The parameter is collected normally, by whatever mechanism the implementation uses to gather values.

The condition expression uses the same syntax as body conditions (§5) and references top-level parameters by their bare, **uppercase** name (e.g. `CREDIT_CARD_TYPE`, not `[[[CREDIT_CARD_TYPE]]]`). At build time the condition is evaluated against the values collected or defaulted so far, in declaration order.

To avoid circular or ambiguous evaluation, an `exclude_condition` may only reference parameters declared **before** the one carrying the condition. A forward or self-reference is a parse error (§9.5). The condition expression itself is parsed and validated at parse time, so syntax errors are reported before any build begins.

```
params:
  credit_card_type:
    type: option_single
    opts:
      - "visa"
      - "mastercard"
    def: "visa"

  visa_card_expiry_date:
    type: string
    desc: Expiry date for Visa card
    exclude_condition: CREDIT_CARD_TYPE != "visa"
    def: "12/25"
```

In this example, `visa_card_expiry_date` is **hidden** (excluded from the build) when `credit_card_type` is not `"visa"` — i.e. the condition `CREDIT_CARD_TYPE != "visa"` is truthy. When the card type *is* `"visa"`, the condition is falsy and the parameter is shown (asked). In other words, the expiry date is only collected for Visa cards.

Rules:

- `exclude_condition` is only allowed on **top-level** (root) parameters. Declaring it on a nested `ofields` entry is a parse error.
- `exclude_condition` is not allowed on `object_shape` variables (they are never asked at build time, so a condition would be meaningless).
- Variable references in a condition MUST be uppercase (§4.1) and MUST name a previously-declared top-level parameter.
- A parameter hidden by its condition still receives its `def` default value for rendering — it is simply not collected, by any means, while hidden.
- The condition expression is parsed and validated at parse time (§9 step 5a).

---

4. Prompt Body Syntax

The prompt body is free text that may contain three kinds of dynamic constructs:

4.1 Variable Placeholders

A variable reference is written as `[[[VAR_NAME]]]`. Variable references MUST be written in **uppercase**. The matching against the variable name declared in `params` (which stays lowercase) is case-insensitive, so a variable declared as `username` is referenced as `[[[USERNAME]]]`. Using a lowercase or mixed-case reference (e.g. `[[[name]]]`, `[[[UserName]]]`) is a parse error.

```
Hello, [[[USERNAME]]]!
```

4.1.1 List

A `list` variable referenced as `[[[VAR]]]` renders its elements joined with `", "`. The rendering of each element depends on its `etype`:

- For scalar `etype`s (`string`, `long_string`, `number`, `boolean`), each element is rendered as its literal value (see §4.6 for value rendering rules).
- For an `object` `etype` (declared inline with its own `ofields`, or via a referenced `object_shape` per §3.4), each element is rendered as a comma-separated list of `field: value` pairs (no enclosing braces), e.g. `name: free will, field: determinism`. For structured output, prefer dotted-path access (§4.1.2), field projection (§4.1.5), or a `for` loop (§4.3).

```
params:
  topics:
    type: list
    etype: string
    def: ["ethics", "logic", "metaphysics"]
  years:
    type: list
    etype: number
    def: [-384, 1711, 1905]
  debatable:
    type: list
    etype: boolean
    def: [true, false, true]
  concepts:
    type: list
    etype: object
    ofields:
      name:
        type: string
        def: "free will"
      field:
        type: string
        def: "determinism"
    def:
      - { name: "free will", field: "determinism" }
      - { name: "justice", field: "ethics" }
--
Topics:     [[[TOPICS]]]
Years:      [[[YEARS]]]
Debatable:  [[[DEBATABLE]]]
Concepts:   [[[CONCEPTS]]]
```

renders to:

```
Topics:     ethics, logic, metaphysics
Years:      -384, 1711, 1905
Debatable:  true, false, true
Concepts:   name: free will, field: determinism, name: justice, field: ethics
```

For structured element access, iterate the list with a `for` loop (see §4.3) and use a dotted path on the loop variable to print a single field:

```
Concepts:
{{{for CONCEPT in CONCEPTS}}}
- [[[CONCEPT.NAME]]]
{{{end for}}}
```

renders to:

```
Concepts:
- free will
- justice
```

4.1.2 Object Field Access (Dotted Paths)

Fields of an `object` variable are referenced with a **dotted path** inside the placeholder: `[[[<OBJECT>.<FIELD>]]]`. Paths may chain through any number of nested objects, so a field declared at arbitrary depth is reached as

`[[[OBJ1.OBJ2.OBJ3.FIELDNAME]]]`. Every path segment MUST be uppercase; segments resolve case-insensitively against the `ofields` declared in `params` (which stay lowercase).

```
params:
  theory:
    type: object
    ofields:
      name:
        type: string
        def: "determinism"
      origin:
        type: object
        ofields:
          era:
            type: string
            def: "ancient"
          contested:
            type: boolean
            def: true
--
Theory:    [[[THEORY.NAME]]]
Origin:    [[[THEORY.ORIGIN.ERA]]]
Contested: [[[THEORY.ORIGIN.CONTESTED]]]
```

Dotted paths also work inside loop bodies, where the leading segment is the loop variable (see §4.3) bound to the current list element. The loop variable MUST be uppercase:

```
{{{for ARGUMENT in ARGUMENTS}}}
- [[[ARGUMENT.PREMISE]]] => [[[ARGUMENT.CONCLUSION]]]
{{{end for}}}
```

4.1.3 Option Single

An `option_single` variable holds exactly one value chosen from its `opts`. The optional `etype` (default `string`) determines the kind of each option; the supported etypes are `string`, `long_string`, `number`, the inline `object`, and a referenced `object_shape` (§3.4) — either object-shaped form requires `label` (§3.6). Referencing the variable as `[[[VAR]]]` renders the selected option's value: scalars render verbatim (see §4.6), and an object option renders as a comma-separated `field: value` list with no enclosing braces (use dotted-path access, §4.1.2, to extract fields). When no value is supplied, the `def` default is used.

```
params:
  env:
    type: option_single
    desc: Target environment
    opts:
      - "development"
      - "staging"
      - "production"
    def: "production"
--
Deploying to [[[ENV]]].
```

renders (with default) to:

```
Deploying to production.
```

A number-etype example:

```
params:
  port:
    type: option_single
    etype: number
    opts:
      - 80
      - 443
      - 8080
    def: 443
--
Listening on port [[[PORT]]].
```

renders (with default) to:

```
Listening on port 443.
```

4.1.4 Option Multi

An `option_multi` variable holds zero or more values chosen from its `opts`. The required `etype` determines the kind of each option; the supported etypes are `string`, `long_string`, `number`, the inline `object`, and a referenced `object_shape` (§3.4) — either object-shaped form requires `label` (§3.6). Referencing the variable as `[[[VAR]]]` renders the selected values joined with `", "`, in the order they were chosen. An empty selection renders as an empty string. When `etype` is `object` (inline or via an `object_shape`), each chosen option is an object; use dotted-path access inside a loop or projection to extract fields.

```
params:
  features:
    type: option_multi
    desc: Feature flags to enable
    opts:
      - "auth"
      - "logs"
      - "metrics"
      - "cache"
    def: ["auth", "metrics"]
--
Enabled features: [[[FEATURES]]]
```

renders (with default) to:

```
Enabled features: auth, metrics
```

4.1.5 List Field Projection

When a dotted path traverses into a `list` of `object` elements, the next segment is **projected** across every element of the list. The result is the list of that field's values from each element, joined with `", "`. This lets a single placeholder expand a whole column of a list-of-objects without a loop:

```
params:
  model:
    type: object
    ofields:
      fields:
        type: list
        etype: object
```

```
        ofields:
          name:
            type: string
        def:
          - { name: "username" }
          - { name: "email" }
--
const { [[[MODEL.FIELDS.NAME]]] } = req.body;
```

renders (with default) to:

```
const { username, email } = req.body;
```

4.2 Conditional Expressions (Ternary)

A conditional is written as `{{{condition ? value_if_true : value_if_false}}}`.

```
{{{AGE >= 18 ? "adult" : "minor"}}}
```

Conditions reference variables by their **bare, uppercase** name (`VAR`, not `[[[VAR]]]`). The `[[[...]]]` delimiters are reserved for placeholders that print a value into the prompt body (§4.1) and for ternary branch value references; they are **not** used inside condition expressions. Conditions may also use literal values and any of the operators in §5. String literals may be written with either double (`"..."`) or single (`'...'`) quotes; both yield the same value. A quote character that does not start a string literal must be escaped by enclosing the literal in the other quote style.

4.3 For Loops

A loop iterates over a list-valued variable — i.e. a `list` variable or an `option_multi` variable — and repeats its body once per element.

```
{{{for ENDPOINT in ENDPOINTS}}}
- [[[ENDPOINT.METHOD]]] [[[ENDPOINT.PATH]]]
{{{end for}}}
```

- `{{{for <ITEM> in <LIST>}}}` opens the loop. `<ITEM>` is the loop variable binding and MUST be uppercase; `<LIST>` may be any (all-uppercase) dotted path that resolves to a list — a bare variable name (`ENDPOINTS`), a nested field (`MODEL.ITEMS`), or a projected list (`MODEL.FIELDS.NAME`, §4.1.5) — written without `[[[...]]]` wrapping, since `[[[...]]]` is reserved for printing values into the prompt body. For backward compatibility, `[[[VAR]]]` wrapping is still tolerated and stripped.
- `{{{end for}}}` closes the loop.
- Inside the body, the current item is referenced by `<ITEM>.<FIELD>` (both segments uppercase).

4.4 If Blocks (Conditional Blocks)

A conditional block renders its content only when the condition is **truthy** (see §4.6.2 for the truthiness rules). The condition may be any expression from §5 — a bare variable, a comparison, a string test, or a combination using `and` / `or` / `not` and parentheses (§5.1, §5.2) — written exactly as in a ternary condition (§4.2): bare, uppercase variable references, without `[[[...]]]` wrapping.

```
{{{if INCLUDE_AUTH}}}
Authorization: [[[ENDPOINT.HEADERS.AUTHORIZATION]]]
```

```
{{{end if}}}
```

4.5 Escaping and Safety

Only `[[[` and `{{{` trigger expansion. An opening `[[[` without a matching closing `]]]`, or a bare `{{{` without a matching `}}}`, is emitted verbatim, so code snippets containing similar-looking sequences are not misinterpreted. However, recognized block constructs — `{{{for ...}}}`, `{{{end for}}}`, `{{{if ...}}}`, `{{{end if}}}` — MUST be balanced: an unclosed `for` / `if`, or a stray `{{{end for}}}` / `{{{end if}}}` with no matching opener, is a parse error.

A **matched** triple-delimiter group is never emitted verbatim on its own: a matched `{{{..}}}` MUST be a valid ternary/`for`/`if` construct (otherwise it's a parse error), and a matched `[[[..]]]` is always interpreted as a placeholder — regardless of what's inside. This matters because prompts routinely need to write these exact sequences literally (Mustache/Handlebars' unescaped-interpolation syntax is `{{{value}}}`, for instance).

To write either sequence literally, escape its **opening** delimiter with a backslash: `\{{{` renders as `{{{`, and `\[[[` renders as `[[[`, with the backslash consumed and no construct/placeholder parsing attempted. No escape is needed for the closing `}}}` / `]]]` — once the opening delimiter has been escaped no construct is opened, so the later `}}}` / `]]]` is simply literal text (the same way an unmatched one already is). Scanning resumes right after the escaped opener, so a real placeholder or construct later on the same line is still expanded: `\[[[ literal ]]] then [[[NAME]]]` renders the first group literally and substitutes the second. An escaped opener inside a `for` / `if` body is likewise literal and never opens or closes a block. A backslash not immediately followed by `{{{` or `[[[` has no special meaning and is emitted as-is — so `\\{{{` is a literal `\` followed by an escaped `{{{`, and things like `\n`, `\t`, or a bare `\` elsewhere in the body are untouched (there is no general backslash-escape-sequence syntax; only these two delimiter escapes exist).

```
Mustache's unescaped syntax looks like \{{{value}}}.
```

renders to:

```
Mustache's unescaped syntax looks like {{{value}}}.
```

4.6 Value Rendering and Truthiness

4.6.1 Value Rendering

When a value is substituted into the prompt body (via `[[[VAR]]]`, a ternary branch, or list joining), it is rendered as a string according to its type:

Type	Rendering
`string`	The string verbatim.
`long_string`	The string verbatim (may contain newlines).
`number`	Integer-valued numbers render without a fractional part (e.g. `80`); otherwise the full floating-point representation is used (e.g. `3.14`). Negative numbers are prefixed with `-`.
`boolean`	`true` or `false`.
`list`	Elements rendered per this table, joined with `", "`.
`object`	`field: value` pairs (each `value` rendered per this table), joined with `", "`, **without** enclosing braces. Field order follows declaration order.

4.6.2 Truthiness

Conditions in `if` blocks, ternaries, and the operand of `!` are evaluated for truthiness as follows:

Type	Truthy when
`boolean`	the value is `true`
`string` / `long_string`	the string is non-empty
`number`	the value is non-zero
`list`	the list is non-empty
`object`	the object has at least one field

A bare variable reference used as a condition (e.g. `{{{if FLAG}}}`) is truthy per the table above.

4.7 Whitespace Around Block Tags

Exactly one newline is trimmed immediately after each of the four block-tag delimiters — the opening `{{{for ...}}}` and `{{{if ...}}}` tags, and the closing `{{{end for}}}` and `{{{end if}}}` tags — so that writing a loop or if-block on its own line does not introduce a blank line into the rendered output:

- If the character(s) immediately following a tag's closing `}}}` are a single newline (`\n` or `\r\n`), that newline is consumed and does **not** appear in the output. At most one newline is trimmed per tag; any further blank lines are preserved as-is.
- No other whitespace is trimmed: leading spaces/tabs before a tag, and a tag not immediately followed by a newline, are left untouched.
- Ternary expressions (`{{{cond ? a : b}}}`, §4.2) and placeholders (`[[[VAR]]]`, §4.1) consume no surrounding whitespace at all — everything before and after them, including newlines, is preserved verbatim.

For example:

```
Servers:
{{{for SERVER in SERVERS}}}
- [[[SERVER]]]
{{{end for}}}
Done.
```

renders (for `SERVERS = ["a", "b"]`) to:

```
Servers:
- a
- b
Done.
```

— not to a version with a blank line after `Servers:`, before each `- [[[SERVER]]]` line, or before `Done.`, which is what a naive line-for-line reading of the template would otherwise produce. An independent implementation that skips this rule will disagree with every rendered example in this document.

---

5. Condition Operators

All binary operators are **left-associative**; `!` and `not` are prefix unary operators. The ternary `? :` (§5.2) is neither — its branches are plain values, not nested conditions, so it doesn't chain and associativity doesn't apply to it (see §5.2). Type checking is enforced at runtime; for example, comparing a number to a string fails evaluation. String literals may use either single (`'...'`) or double (`"..."`) quotes interchangeably. Variables in conditions are referenced by their **bare, uppercase** name (e.g. `A`, `FLAG`, `TAGS`) — **not** wrapped in `[[[...]]]`, which is reserved for placeholders in the prompt body (§4.1) and ternary branch value references.

Operator	Meaning	Example
`=`	Equal (`==` is accepted as an alias)	`A = B`
`!=`	Not equal	`X != 'test'`
`!`	Logical NOT (unary)	`!FLAG`
`contains`	String contains / list membership	`TEXT contains "hello"` ; `TAGS contains "api"`
`starts_with`	String starts with	`PATH starts_with "/home"`
`ends_with`	String ends with	`EXT ends_with ".js"`
`>=`	Greater or equal (numbers)	`COUNT >= 5`
`>`	Greater than (numbers)	`AGE > 18`
`<=`	Less or equal (numbers)	`SCORE <= 100`
`<`	Less than (numbers)	`PRICE < 10`

Notes:

- `==` is tokenized as `=`; the two are interchangeable.
- `contains` is overloaded on the type of its **left** operand, which is the only operand ever checked for list-ness:
  - If the left operand is a `list`, `contains` performs membership testing (whether the list contains the right operand, compared by value) — e.g. `TAGS contains "api"`. The right operand must be a single element — `string`, `number`, `boolean`, or `object` (compared by value) — and must **not** itself be a `list`; `TAGS contains OTHER_LIST` is a type error.
  - If the left operand is a `string` (or `long_string`), `contains` tests whether the left string contains the right string as a substring — e.g. `TEXT contains "hello"`. The right operand must also be a `string` / `long_string`; anything else, including a `list`, is a type error (so the list must be the left operand: `"api" contains TAGS` is a type error, not membership testing).
  - Any other left-operand type (`number`, `boolean`, `object`) is always a type error for `contains`.

  `starts_with` and `ends_with` apply only to strings (both operands).
- Comparison operators (`>`, `<`, `>=`, `<=`) require both operands to be numbers; a type mismatch fails evaluation.
- `=` and `!=` require operands of the same scalar kind (number/number, string/string-or-long_string, boolean/boolean); a mismatch fails evaluation.
- The string operators (`contains`, `starts_with`, `ends_with`) may also be written in method-call form: `VAR.contains("x")`, `VAR.starts_with("x")`, `VAR.ends_with("x")`. This form is rewritten to the infix form before evaluation and is equivalent.

5.1 Logical Combinators (`and`, `or`, `not`)

Comparisons and string tests (above) may be combined into a compound condition with the keyword operators `and`, `or`, and `not`:

Operator	Meaning	Example
`not`	Logical NOT (unary, keyword form)	`not (TIER = "free" or TIER = "trial")`
`and`	Logical AND (binary)	`HOURS > 10 and TAGS contains "api"`
`or`	Logical OR (binary)	`TIER = "pro" or TIER = "enterprise"`

`and` and `or` do **not** require their operands to be `boolean`-typed: each operand is evaluated for **truthiness** (§4.6.2), exactly as an `if` block or bare-variable condition is, and the result is always a `boolean`. Both are **short-circuiting**: for `and`, the right operand is evaluated (and its value looked up) only if the left operand is truthy; for `or`, only if the left operand is falsy. This matters when the right operand would otherwise fail to resolve — `HAS_TAGS and TAGS contains "api"` never touches `TAGS` when `HAS_TAGS` is falsy.

`not` is a keyword alias for `!` with **different precedence** (§5.2): `!` binds only to the single primary immediately after it (a variable, literal, or parenthesized group), matching its existing tight-binding behavior, while `not` binds to the entire comparison/equality/string-operator expression that follows — e.g. `not A contains "x"` is `not (A contains "x")`, whereas `!A contains "x"` is `(!A) contains "x"`. Prefer `not` when combining with `and` / `or`; `!` remains available for negating a single value inline (e.g. `!FLAG`).

5.2 Parenthesized Grouping and Operator Precedence

Parentheses `( ... )` group any sub-expression built from the operators in §5 and §5.1, overriding the default precedence below. A group's contents are themselves a full condition expression, so groups may nest and may contain `and`, `or`, and `not`.

Parentheses are **optional**: a condition consisting of a single comparison, string test, or bare variable/literal (e.g. `HOURS > 10`, `FLAG`) never needs to be wrapped, and default precedence already resolves an unparenthesized compound expression unambiguously (e.g. `A or B and C` parses as `A or (B and C)`, per the precedence order below). Parentheses are only necessary to force a grouping that default precedence would not otherwise produce, e.g. `(A or B) and C`.

From highest to lowest precedence:

1. `!` (unary NOT, binds to a single primary — a variable, literal, or parenthesized group)
2. `>`, `<`, `>=`, `<=`
3. `=`, `!=`
4. `contains`, `starts_with`, `ends_with`
5. `not` (unary NOT, binds to the comparison/equality/string-operator expression that follows)
6. `and`
7. `or`
8. `? :` ternary (lowest precedence)

The ternary remains the lowest-precedence operator. Note: branches of a ternary are plain values (a `[[[VAR]]]` reference, a quoted string, a bare number/boolean, or literal text); **nested ternaries are not supported** inside ternary branches — this is unaffected by `and` / `or` / `not` /parentheses, which apply only to the condition itself. It is not associative (there is nothing to associate — see above), so writing a second `? :` inside a branch does not chain into a nested conditional; that text is treated as part of the plain-value branch and rendered **literally**, verbatim, whichever branch is taken. For example, `{{{A = "x" ? "one" : A = "y" ? "two" : "three"}}}` with `A = "z"` renders the literal text `A = "y" ? "two" : "three"`, not an evaluated nested ternary.

```
{{{if (TIER = "pro" or TIER = "enterprise") and not SUSPENDED}}}
Full access enabled.
{{{end if}}}
```

---

6. Features

Feature	Supported
String & long string variables	Yes
Number, boolean, list, option_single, option_multi	Yes
Nested `object` variables (with `ofields` block)	Yes
`object_shape` type definitions (reusable shape, not asked)	Yes
Variable defaults (`def`)	Yes
Option lists (`opts`)	Yes
Nested `etype` for lists / option_multi	Yes
Object type reuse via `etype: <object_shape>` references (§3.4)	Yes
Shape reuse via `type: <object_shape_name>` on `object` /fields (§3.4.2)	Yes
Inline `etype: object` (with inline `ofields`)	Yes
`[[[VAR]]]` placeholder syntax	Yes
`[[[OBJ.FIELD]]]` dotted-path object field access	Yes
`[[[OBJ1.OBJ2.OBJ3.FIELD]]]` arbitrary-depth nesting	Yes
`[[[LIST.FIELD]]]` list-of-objects field projection	Yes
`{{{cond ? a : b}}}` ternary condition	Yes
All operators listed in §5 (incl. `==` alias, method-call form)	Yes
`{{{for <ITEM> in <LIST>}}}<content>{{{end for}}}` loops	Yes
`{{{if <COND>}}}<content>{{{end if}}}` conditional blocks	Yes
Complex conditionals with variables and literals	Yes
Runtime type checking during evaluation	Yes
Code snippets containing an unmatched, near-miss sequence like `[[[...]]]` (two closing brackets, not three — deliberately not a valid placeholder)	Yes
Recursive object definitions (objects inside objects)	Yes
`source` provenance metadata field (§2.1)	Yes
Build-time `exclude_condition` on parameters (§3.7)	Yes

---

7. Data Model (Conceptual)

The following describes the conceptual structure of a parsed UPL document. Field names mirror the metadata keys in the file.

7.1 Prompt

Field	Type	Description
`name`	string	The prompt identifier. MUST match the file's base name and be lowercase alphanumeric (UTF-8) + underscores only (see §2).
`title`	string (optional)	Human-readable title.
`desc`	string (optional)	Description text.
`source`	string (optional)	Provenance `<host>/<username>/<prompt_name>` for prompts pulled from a repository (§2.1). Absent for locally authored prompts.
`prompt`	string	The raw prompt body (the source text before placeholder substitution). Rendering is performed separately by the builder.
`variable_definitions`	map	Declared input variables.
`variable_defaults`	map	Default values keyed by variable name (dotted path for nested fields).
`template`	Template	Parsed body AST (see §7.6). Not serialized; reconstructed from `prompt` on load.

7.2 VariableDefinition

Field	Type	Description
`type`	VariableType	One of the types in §3.1.
`desc`	string (optional)	Description.
`options`	list (optional)	Allowed options (for `option_single` / `option_multi`). Each entry matches `element_type`.
`element_type`	VariableType (optional)	Element type (for `list` / `option_single` / `option_multi`). When `etype` names a declared `object_shape` variable (§3.4), this is `object` and `ofields_definitions` holds the referenced object_shape's resolved fields.
`element_ref`	string (optional)	Name of the declared `object_shape` variable referenced via `etype: <name>` (resolved at parse time; kept for downstream default synthesis).
`label`	string (optional)	For `option_single` / `option_multi` with an `object_shape` etype: the field whose value is the menu label (§3.6).
`type_ref`	string (optional)	Name of the declared `object_shape` variable whose `ofields` an `object` (or nested object field) reuses via `type: <name>` (resolved at parse time into `ofields_definitions`); `type` is then `Object`.

Field	Type	Description
`ofields_definitions`	map (optional)	Object fields (for `object` / `object_shape`, or resolved from an `object_shape` reference).
`exclude_condition`	CondExpr (optional)	Build-time condition (§3.7). When truthy at build time, the parameter is hidden (excluded from the build). Only on top-level parameters; not on `object_shape`.

7.3 Value

A value is one of:

- **String** — plain string.
- **LongString** — multi-line string. (Rendered identically to String; the distinction is preserved so authoring tools can offer multi-line input.)
- **Number** — floating-point.
- **Boolean** — true/false.
- **List** — ordered list of values.
- **Object** — ordered map of string → value. Iteration and rendering preserve field declaration order.

7.4 Conditional Expression

A condition is represented as an expression tree:

- **Binary** — `left op right` (operators per §5)
- **Unary** — `op expr` (e.g. `!flag`)
- **Literal** — a constant value.
- **Variable** — a reference to a named variable.

7.5 ForLoop

Field	Type	Description
`item`	string	Name of the loop variable.
`list`	string	Name of the list-valued variable being iterated (a `list` or `option_multi`, possibly a dotted path — §4.3, §6).
`body`	list	Body nodes rendered once per element (see §7.6).

(Matches the `Loop { item, list, body }` `Node` variant in §7.6 exactly — this is the same node, described here at the field level.)

7.6 Template / Node

The parsed body is a tree of nodes (`Template { nodes: Vec<Node> }`). A node is one of:

- **Text(string)** — literal text emitted verbatim.
- **Placeholder(string)** — a `[[[VAR]]]` or `[[[OBJ.FIELD]]]` reference, resolved and rendered per §4.1 / §4.6.
- **Ternary { cond, true_branch, false_branch }** — `{{{cond ? a : b}}}`; `cond` is a Conditional Expression (§7.4); branches are strings rendered per §4.6.
- **Loop { item, list, body }** — a `for` loop (§4.3); `body` is a `Vec<Node>`.
- **If { cond, body }** — an `if` block (§4.4); `body` is a `Vec<Node>`.

8. Examples

8.1 Ask for a Philosophical Debate Outline

```
--
name: ask_debate_outline
title: Ask for a Philosophical Debate Outline
desc: Ask the assistant to outline a debate from a list of positions
params:
  positions:
    type: list
    desc: List of positions to include in the debate
    etype: object
    ofields:
      stance:
        type: option_single
        desc: Whether the position is for or against
        opts:
          - "for"
          - "against"
          - "neutral"
        def: "for"
      claim:
        type: string
        desc: The claim being argued
        def: "free will exists"
      reasoning:
        type: long_string
        desc: Supporting reasoning (if any)
        def: "none"
      sources:
        type: object
        desc: Source attribution
        ofields:
          author:
            type: string
            def: "anonymous"
          year:
            type: string
            def: "unknown"
        def: {}
  include_counterpoints:
    type: boolean
    desc: Whether counterpoints should be requested for each position
    def: true
--
Please write a debate outline covering the following positions:

{{{for POSITION in POSITIONS}}}
- [[[POSITION.STANCE]]] [[[POSITION.CLAIM]]] (reasoning: [[[POSITION.REASONING]]])
{{{if INCLUDE_COUNTERPOINTS}}}
  Note: provide a counterpoint to this position.
{{{end if}}}
{{{if POSITION.REASONING != "none"}}}
  Note: this position includes explicit reasoning.
{{{end if}}}
{{{end for}}}

Explain how each position relates to the central question.
--
```

8.2 Ask for a Study Plan

```
--
name: ask_study_plan
title: Ask for a Study Plan
desc: Ask the assistant to recommend a study plan for a given subject
params:
  subject:
    type: option_single
    desc: Subject to study (ethics, logic, metaphysics)
    opts:
      - "ethics"
      - "logic"
      - "metaphysics"
    def: "ethics"
  hours:
    type: number
    desc: Hours available per week
    def: 5
  prefer_grounded:
    type: boolean
    desc: Whether the caller prefers historically grounded material
    def: true
--
I want to study [[[SUBJECT]]] and have about [[[HOURS]]] hours per week.

{{{HOURS > 10 ? "I have ample time, so a deep, structured curriculum matters." : "I have limited time, so focus a

{{{PREFER_GROUNDED ? "Please ground the plan in primary sources." : "Please use accessible modern overviews."}}}

Describe the best way to approach this subject and explain why.
--
```

8.3 Ask for a Theoretical Framework Review

```
--
name: ask_theory_review
title: Ask for a Theoretical Framework Review
desc: Ask the assistant to review and explain a theoretical framework
params:
  framework:
    type: object
    desc: Full theoretical framework to review
    ofields:
      domain:
        type: string
        desc: Domain of inquiry
        def: "epistemology"
      influence:
        type: number
        desc: Estimated influence score (0-100)
        def: 75
      origin:
        type: object
        desc: Origin details
        ofields:
          tradition:
            type: option_single
            opts:
              - "rationalist"
              - "empiricist"
              - "pragmatist"
            def: "rationalist"
          founder:
            type: string
            desc: Principal founder
```

```
            def: "Descartes"
          era:
            type: string
            def: "17th century"
          rival:
            type: string
            def: "Locke"
      scope:
        type: object
        desc: Scope of application
        ofields:
          breadth:
            type: number
            def: 3
          depth:
            type: number
            def: 5
        def: { breadth: 3, depth: 5 }
--
Please review the following theoretical framework and tell me whether it is sound and well-scoped:

- Domain: [[[FRAMEWORK.DOMAIN]]]
- Influence (0-100): [[[FRAMEWORK.INFLUENCE]]]
- Tradition: [[[FRAMEWORK.ORIGIN.TRADITION]]]
- Founder: [[[FRAMEWORK.ORIGIN.FOUNDER]]]
- Scope breadth: [[[FRAMEWORK.SCOPE.BREADTH]]]
- Scope depth: [[[FRAMEWORK.SCOPE.DEPTH]]]

Point out any weaknesses (for example, overreach beyond its domain) and suggest improvements.
--
```

8.4 Ask for a Philosopher Roster (object_shape reuse)

This example shows the difference between `object` (asked to the user) and `object_shape` (a reusable shape, never asked on its own). `philosopher` is an `object_shape` reused by the `thinkers` list and by the `focal` object (via `type: philosopher`).

```
--
name: ask_philosopher_roster
title: Ask for a Philosopher Roster
desc: Collect a list of philosophers plus a focal philosopher, all sharing one shape
params:
  philosopher:
    type: object_shape
    ofields:
      name:
        type: string
        def: "Socrates"
      era:
        type: number
        def: -470
  thinkers:
    type: list
    etype: philosopher
    def:
      - { name: "Plato", era: -428 }
      - { name: "Aristotle", era: -384 }
  focal:
    type: philosopher
    def: { name: "Plato", era: -428 }
--
Focal: [[[FOCAL.NAME]]] (b. [[[FOCAL.ERA]]])
All:
{{{for P in THINKERS}}}
- [[[P.NAME]]] (b. [[[P.ERA]]])
{{{end for}}}
--
```

At build time the builder prompts for `thinkers` (a list whose each item is collected using the `philosopher` object_shape shape) and for `focal` (an object reusing the `philosopher` shape via `type: philosopher`). It never prompts for `philosopher` on its own.

---

9. Parsing and Validation

A conforming UPL implementation MUST perform the following steps:

1. **Parse** — split the file into metadata and body sections using the `--` delimiter (the leading `--` is optional). The body ends at the first line that is exactly `--`, if any (§2); any non-blank line found after it is a parse error. Parse `def` / `opts` values per the literal syntax in §3.3.1 (including the `long_string` heredoc form, §3.5).
2. **Validate header** — ensure required metadata fields are present and well-formed. In particular, `name` is required, MUST contain only lowercase alphanumeric (UTF-8) characters and underscores, and MUST equal the file's base name (the file name with its `.txt` / `.upl` extension — and a single trailing `.prompt` segment, if present — stripped). The file MUST use the `.txt` or `.upl` extension. `source` (§2.1), if present, is accepted as informational metadata.
3. **Validate params** — ensure every variable declaration has a valid `type` and that the type-specific fields (`etype` , `ofields` , `opts` , `label`) are used only where permitted (§3.3) (note: `type: <object_shape_name>` is not a separate key — it is the `type` value naming a declared object_shape). Resolve element references (§3.4) and verify they point to declared `object_shape` variables with `ofields` , reporting any cycle. Resolve `type: <object_shape_name>` references on `object` variables/fields. Reject any by-name `etype` / `type` reference that names a declared `object` (only `object_shape` is referenceable by name).
4. **Validate field types** — for each variable, ensure `def` and `opts` values match the declared `type` and `element_type` (§3.3). A `def` whose `VariableValue` kind does not match the declared type (and, for lists, whose elements do not match `etype`) is a parse error. For `option_single` / `option_multi` , also ensure the `def` is one of the declared `opts` (every element, for `option_multi`) — a `def` that is not an offered option is a parse error (§3.3).
5. **Validate body** — parse the body into a Template (§7.6); ensure every variable reference is written in uppercase (§4.1); ensure `for` / `if` blocks are balanced (§4.5). For each dotted-path reference whose root names a declared variable (or an in-scope loop variable), verify that every segment after the root names a real field of the referenced object's resolved shape (an unknown field is a parse error). A root variable that is not declared in `params` is **not** a parse error: its value may be supplied programmatically at render time (see the `[[[URL]]]` example in §3.5), and existence of a *value* is enforced at render time (step 6) as `MissingValue` . 5a. **Validate conditions** — for each top-level parameter that declares an `exclude_condition` (§3.7), verify that the condition expression is syntactically valid (parsed with the §5 condition syntax), that every variable reference inside it is uppercase (§4.1), and that every referenced variable is a top-level parameter declared **before** the one carrying the condition (forward and self-references are parse errors). Reject an `exclude_condition` on an `object_shape` variable or on a nested `ofields` entry.
6. **Render** — substitute variables, evaluate conditionals with runtime type checking (§5), and expand loops, using the rendering and truthiness rules in §4.6. A reference to a variable for which no value was supplied fails here with `MissingValue` ; a `for` loop over a non-list value fails here as well. At build time, before collecting or accepting a value for a parameter with an `exclude_condition` , evaluate the condition against the values collected or defaulted so far (in declaration order); a truthy condition hides (excludes) the parameter from the build (§3.7).

Errors raised at any step SHOULD include the offending field name, line number, and a clear description of the failure.

---

10. Conformance

An implementation conforms to this standard if it:

- Parses files with the `.txt` or `.upl` extension in the format described in §2 (leading `--` optional).
- Enforces that the `name` metadata field is present, lowercase alphanumeric (UTF-8) plus underscores only, and matches the file's base name (a single trailing `.prompt` segment stripped if present).
- Accepts the optional `source` metadata field (§2.1) without affecting parsing/rendering.

- Supports all variable types in §3.1 with their associated validation rules (§3.3), including the literal value syntax in §3.3.1 and the `long_string` heredoc form (§3.5).
- Rejects `def` values whose kind does not match the declared `type` / `etype` (§3.3) as parse errors, and `option_single` / `option_multi` `def` values that are not among the declared `opts` (§3.3).
- Rejects tab characters in the indentation of the metadata section (§2), whatever column the tab-indented line would otherwise land on.
- Supports object type reuse via `etype: <object_shape>` references (§3.4) and the `label` field for object_shape-etype options (§3.6). Supports `type: <object_shape_name>` shape reuse on `object` variables/fields (§3.4.2). Supports the inline `etype: object` (with inline `ofields`). Treats `object` params as collectible (asked at build time) and `object_shape` params as pure type definitions (never asked at their definition site, only at reference sites).
- Expands `[[[VAR]]]` placeholders (including dotted-path access and list field projection), `{{{cond ? a : b}}}` ternaries, `{{{for ... in ...}}}{{{end for}}}` loops (over `list` and `option_multi`), and `{{{if ...}}}{{{end if}}}` blocks.
- Renders values and evaluates truthiness per §4.6. Object fields are rendered and iterated in **declaration order** (§4.6.1, §7.3).
- Trims exactly one newline immediately after each `{{{for ...}}}`, `{{{end for}}}`, `{{{if ...}}}`, and `{{{end if}}}` tag (§4.7); no other construct trims surrounding whitespace.
- Validates, at parse time, that every dotted-path segment after a declared root names a real field of the referenced object's resolved shape (§9 step 5); an unknown field on a declared object is a parse error. A root variable not declared in `params` is allowed at parse time and resolved at render time (§9 step 6).
- Enforces runtime type checking for all operators in §5, including the `contains` list-membership overload and the `==` alias for `=`.
- Applies the `and` / `or` / `not` short-circuit and truthiness semantics of §5.1, and the operator precedence in §5.2.
- Reports `for` / `if` block imbalance as parse errors (§4.5), and honors the `\{{{` / `\[[[` escapes for literal delimiter text (§4.5).
- Supports the `exclude_condition` field on top-level parameters (§3.7): parses and validates condition expressions at parse time (§9 step 5a), evaluates them at build time to hide (exclude) parameters whose condition is truthy, and rejects any external override for a hidden parameter, regardless of the value-supply mechanism.
- Reports errors as described in §9.